Graph Neural Networks (GNN) for Image Processing

Table of Contents

1. Introduction to Graph Neural Networks 3
   1.1 What is a Graph? 3
   1.2 What is a Graph Neural Network (GNN)? 3
   1.3 Why Use GNNs in Image Processing? 3
2. Core Concepts You Need Before Starting 4
   2.1 How a Graph Is Represented for PyTorch Geometric 4
   2.2 Message Passing (the Core Idea of a GNN Layer) 4
   2.3 Common GNN Layers 4
   2.4 A Note on Over-Smoothing ..... **Error! Bookmark not defined.**
3. Applications of GNNs in Image Processing 5
4. Environment Setup 6
5. General Step-by-Step Methodology 7
   Step 1: Define the Problem and Choose a Dataset 7
   Step 2: Preprocess the Images 7
   Step 3: Convert Each Image into a Graph 7
   Step 4: Extract Node Features 7
   Step 5: Construct the Edges (Graph Topology) 8
   Step 6: Build the PyTorch Geometric Data Object 8
   Step 7: Design the GNN Architecture 8
   Step 8: Define the Loss Function and Optimizer 8
   Step 9: Train the Model 8
   Step 10: Evaluate the Model 9
   Step 11: Visualize Results 9
6. Worked Example: Image Classification with Superpixel Graphs + GCN 10
   6.1 Steps 3–5 in Code: Image → Graph Conversion 10
   6.2 Step 6 in Code: Building the Dataset 10
   6.3 Step 7 in Code: GCN Model 11
   6.4 Steps 8–9 in Code: Training Loop 11
   6.5 Step 11 in Code: Visualizing the Graph 12
7. Extending the Pipeline to Other Tasks 13
   7.1 Semantic Segmentation 13

7.2 Object Detection / Scene Graph Generation 13
7.3 Point Clouds / 3D Vision 13
8. Common Errors and Debugging Tips 14
9. Suggested Datasets and Further Resources 15
9.1 Documentation and Background Reading 15
9. Key Papers for Background Reading 15

1. Introduction to Graph Neural Networks

1.1 What is a Graph?

A graph G = (V, E) consists of a set of nodes (also called vertices) V and a set of edges E that connect pairs of nodes. Each node can carry a feature vector that describes its properties, and each edge represents a relationship between two nodes such as spatial adjacency, similarity, or distance.

Images are normally stored as a regular grid of pixels this is called Euclidean data, and it is exactly what convolutional neural networks (CNNs) are designed for. A graph, on the other hand, is non-Euclidean: nodes can have any number of neighbors, and there is no fixed grid structure. Many real image-processing problems are easier to express on top of such a graph.

1.2 What is a Graph Neural Network (GNN)?

A Graph Neural Network is a class of neural networks designed to operate directly on graph-structured data. Instead of sliding a fixed-size convolution kernel over a regular grid (as a CNN does), a GNN repeatedly updates each node's representation by gathering information from its neighboring nodes. This process is called message passing.

After several rounds (layers) of message passing, every node's final representation encodes information not just about itself, but also about its local neighborhood and, as more layers are stacked, about increasingly distant parts of the graph.

1.3 Why Use GNNs in Image Processing?

Although images are naturally grid-structured (which is why CNNs work so well on them), many image processing problems involve relationships that are more naturally represented as graphs:

- **Irregular regions:** Super pixels or segmented regions in an image have irregular shapes and a variable number of neighbors. A graph can represent this naturally, while a grid cannot.
- **Object relationships:** Objects in a scene relate to each other (e.g., “person riding bicycle”, “cup on table”); these relationships form a graph (a scene graph).
- **Non-grid data:** 3D point clouds, human-pose skeletons, and other sensor data do not fit on a regular grid at all, but fit naturally as graphs.
- **Long-range reasoning:** A GNN can connect two distant regions of an image directly through an edge, instead of relying on many stacked convolution/pooling layers to grow the receptive field.

In practice, GNNs are used either as a standalone model on a graph built from an image, or combined with a CNN backbone, where the CNN extracts local features and the GNN reasons about the relationships between regions or objects.

2. Core Concepts You Need Before Starting

2.1 How a Graph Is Represented for PyTorch Geometric

In PyTorch Geometric (PyG), a single graph is stored in a Data object with (at least) the following fields:

- **x** — the node feature matrix, of shape (N, F): N nodes, each described by F numeric features.
- **edge_index** — the graph connectivity, of shape (2, E): each column [source, target] is one directed edge. For an undirected graph, both directions are usually stored.
- **edge_attr** (optional) — edge feature matrix, of shape (E, D).
- **y** — the label(s): a single value per graph for graph-level tasks (e.g., classification), or one value per node for node-level tasks (e.g., segmentation).

2.2 Message Passing (the Core Idea of a GNN Layer)

Every GNN layer updates each node's representation using two steps:

- **Aggregate:** collect the representations of all neighboring nodes N(v) and combine them (e.g., by sum, mean, max, or a weighted/attention-based combination).
- **Update:** combine the aggregated neighbor information with the node's own previous representation, usually through a small neural network (a linear layer + activation function), to produce the new representation.

Stacking K such layers means that, after K layers, every node's final representation depends on its K-hop neighborhood i.e., everything reachable within K edges.

2.3 Common GNN Layers

Layer (PyG class)	Key Idea	When to Use
GCNConv (Graph Convolutional Network)	Normalized averaging of neighbor features.	A simple, fast, general-purpose baseline good first choice for any task.
SAGEConv (GraphSAGE)	Samples and aggregates neighbor features (mean / max / LSTM aggregation).	Larger graphs, or when the graph structure changes between training and testing.
GATConv (Graph Attention Network)	Learns an attention weight for each neighbor, so important neighbors contribute more.	When some neighboring regions/objects are clearly more relevant than others.
GINConv (Graph Isomorphism Network)	Sum aggregation followed by an MLP; theoretically very expressive.	Whole-graph (graph-level) classification tasks.

3. Applications of GNNs in Image Processing

The table below gives a general overview of how GNNs are applied across different image processing tasks. It is meant to help you choose a direction for your assignment.

Application	**How the Image Becomes a Graph**	**GNN Task Type**
Image Classification	Super pixel / region graph: the whole image becomes one graph.	Graph-level classification (one label per image).
Semantic Segmentation	Super pixel or pixel graph; each node gets its own label.	Node-level classification (one label per node).
Object Detection / Scene Graph Generation	Nodes = detected objects (from a detector); edges = candidate relationships between object pairs.	Node classification (object class) + edge classification (relationship type).
Point Cloud / 3D Vision	Nodes = 3D points; edges = k-nearest neighbors in 3D space.	Node-level or graph-level, depending on the task.
Image Retrieval / Matching	Feature graphs are built from key points or regions in each image.	Graph similarity / graph matching.
Medical Imaging (e.g., histopathology, brain connectivity)	Cell / nuclei graphs, or region-of-interest connectivity graphs.	Node-level or graph-level classification.

4. Environment Setup

Set up your Python environment before starting any coding. Google Colab (with the GPU runtime enabled) is recommended for students without access to a local GPU.

```
# Core deep learning library
pip install torch torchvision


# PyTorch Geometric (PyG) and its companion libraries.
# IMPORTANT: the exact install command depends on your torch and CUDA
# version. Check the official PyG installation guide and copy the
# command that matches your environment before running it.
pip install torch_geometric


# Supporting libraries used in this guide
pip install scikit-image networkx matplotlib numpy pandas scikit-learn
```

Suggested installation commands (run in a notebook cell or terminal).

5. General Step-by-Step Methodology

The 12 steps below form a general pipeline that you can adapt to almost any GNN-based image processing task. Steps 1–6 focus on turning images into graphs ("graph construction"). Steps 7–12 focus on building, training, and evaluating the GNN model. Follow them in order each step builds on the previous one.

Step 1 Define the Problem and Choose a Dataset

- Decide the task type: image classification, semantic segmentation, object detection / scene graph, etc.
- For a first attempt, choose a small or medium dataset so that graph construction and training remain manageable (e.g., MNIST, CIFAR-10/100, or a small custom image folder).
- Write down clearly: the input (image), the expected output (label / mask / relationships), and the evaluation metric you will use.

Step 2 Preprocess the Images

- Resize all images to a consistent size.
- Normalize pixel values (e.g., to the range [0, 1], or using the dataset's mean and standard deviation).
- Optionally apply denoising or contrast enhancement if the dataset is noisy.

Step 3 Convert Each Image into a Graph

There are three common strategies for turning an image into a graph:

- **(a) Superpixel-based graph (recommended):** Segment the image into superpixels using SLIC (Simple Linear Iterative Clustering); each superpixel becomes one node. This gives a good balance between graph size and information content, and is the strategy used in the worked example in Section 6.
- **(b) Patch / grid-based graph:** Divide the image into a regular grid of patches; each patch is a node, connected to its neighboring patches. Simple to implement, but less adaptive to image content.
- **(c) Pixel-based graph:** Each pixel is a node. Very fine-grained, but produces enormous graphs only practical for very small images or after heavy downsampling.

Step 4 Extract Node Features

For each node (e.g., each superpixel), compute a feature vector that describes that region. Typical choices include:

- Mean color (RGB or Lab color space) within the region.
- Color variance / standard deviation within the region.
- Texture descriptors (e.g., Local Binary Patterns).
- Normalized centroid position (x, y) of the region within the image.

- A CNN embedding obtained by passing the region (or the whole image, cropped to the region) through a pretrained backbone such as ResNet.

Combine these into a node feature matrix X of shape (N, F), where N is the number of nodes (regions) and F is the number of features per node.

Step 5 Construct the Edges (Graph Topology)

- **Spatial adjacency / Region Adjacency Graph (RAG):** connect two nodes if their corresponding regions share a border in the image. This preserves the spatial layout of the image and is the most common choice.
- **k-Nearest Neighbors (k-NN) in feature space:** connect each node to its k most similar nodes based on feature similarity (e.g., color). This can capture non-local relationships.
- **Combined approach:** use spatial adjacency edges as the main structure, optionally adding a few long-range k-NN edges.
- Optionally, compute edge features such as the difference between the two connected nodes' feature vectors, or the shared-boundary length.

Step 6 Build the PyTorch Geometric Data Object

- For each image, create a torch_geometric.data.Data object containing x (node features), edge_index (edges), and either y (a single graph-level label) or per-node labels, depending on your task.
- Wrap all the per-image Data objects into a Dataset typically by subclassing torch_geometric.data.InMemoryDataset for small/medium datasets.

Step 7 Design the GNN Architecture

- Choose a GNN layer type (GCNConv is a good default see the table in Section 2.3 for alternatives).
- Stack 2–3 GNN layers, each followed by a ReLU activation and (optionally) Dropout.
- **For graph-level tasks (classification):** after the GNN layers, add a global pooling layer (global_mean_pool or global_max_pool) to combine all node representations into a single graph-level vector, followed by one or two fully connected layers as the classifier head.
- **For node-level tasks (segmentation):** skip the pooling step; apply a final linear layer to each node's representation to obtain per-node class scores directly.

Step 8 Define the Loss Function and Optimizer

- **Classification (graph-level or node-level):** use CrossEntropyLoss.
- **Segmentation with class imbalance:** use a weighted CrossEntropyLoss (or a Dice loss) so that rare classes are not ignored.
- **Optimizer:** Adam is a reliable choice a learning rate of 1e-3 with weight decay 5e-4 is a reasonable starting point.

Step 9 Train the Model

- Split the dataset into training, validation, and test sets (e.g., 70% / 15% / 15%).
- Use PyTorch Geometric's DataLoader (from torch_geometric.loader), which automatically batches multiple graphs together and creates the batch vector needed for pooling.
- For each epoch: run a forward pass, compute the loss, run backpropagation, and update the weights with the optimizer. Track the training loss and validation metric after every epoch.

Step 10 Evaluate the Model

- **Classification:** accuracy, precision, recall, F1-score, and a confusion matrix.
- **Segmentation:** pixel accuracy, mean Intersection-over-Union (mIoU), and Dice coefficient.
- **Detection / scene graph:** mean Average Precision (mAP) and relationship recall@K.
- Always compare against a simple baseline for example, a plain CNN, or the same model with the GNN layers removed so you can clearly show what the GNN contributes.

Step 11 Visualize Results

- Plot training and validation loss/accuracy curves over the training epochs.
- Visualize the constructed graph overlaid on the original image (e.g., with NetworkX + Matplotlib), showing nodes at superpixel centroids and edges between adjacent superpixels.
- For segmentation, show predicted masks next to the ground-truth masks.
- For classification, display a few example correct and incorrect predictions with their predicted vs. true labels.

6. Worked Example: Image Classification with Superpixel Graphs + GCN

This section provides a complete, minimal implementation following the steps in Section 5, for an image classification task using superpixel graphs and a GCN. The code is written for PyTorch + PyTorch Geometric and is meant as a template you should adapt the dataset loading, hyperparameters, and number of classes to match your assignment.

6.1 Steps 3–5 in Code: Image → Graph Conversion

This function segments an image into superpixels using SLIC, computes a feature vector for each superpixel (mean Lab color + normalized centroid position), and builds a Region Adjacency Graph (RAG) to define the edges.

```
import numpy as np
import torch
from skimage.segmentation import slic
from skimage.future import graph as skg
from skimage.color import rgb2lab
from torch_geometric.data import Data


def image_to_graph(image, n_segments=75, compactness=10):
    # Step 3: segment the image into superpixels
    segments = slic(image, n_segments=n_segments,
                    compactness=compactness, start_label=0)
    lab_image = rgb2lab(image)
    n_nodes = segments.max() + 1

    # Step 4: build the node feature matrix
    node_features = []
    for seg_val in range(n_nodes):
        mask = segments == seg_val
        mean_color = lab_image[mask].mean(axis=0)          # 3 values
        ys, xs = np.where(mask)
        cx = xs.mean() / image.shape[1]                    # normalized x
        cy = ys.mean() / image.shape[0]                    # normalized y
        node_features.append(np.concatenate([mean_color, [cx, cy]]))
    x = torch.tensor(np.array(node_features), dtype=torch.float)

    # Step 5: build edges using the Region Adjacency Graph (RAG)
    rag = skg.rag_mean_color(image, segments)
    edge_list = []
    for (u, v) in rag.edges():
        edge_list.append([u, v])
        edge_list.append([v, u])                           # undirected
    edge_index = torch.tensor(edge_list, dtype=torch.long).t().contiguous()

    return Data(x=x, edge_index=edge_index), segments
```

image_to_graph(): converts one RGB image into a PyG Data object plus the raw superpixel map (used later for visualization).

6.2 Step 6 in Code: Building the Dataset

Wrap the per-image graphs into an InMemoryDataset so they can be used with PyG's DataLoader.

```
from torch_geometric.data import InMemoryDataset

class ImageGraphDataset(InMemoryDataset):
    def __init__(self, images, labels):
        super().__init__('.')
        data_list = []
        for img, label in zip(images, labels):
            data, _ = image_to_graph(img)
            data.y = torch.tensor([label], dtype=torch.long)
            data_list.append(data)
        self.data, self.slices = self.collate(data_list)


# Example usage:
# train_dataset = ImageGraphDataset(train_images, train_labels)
# val_dataset   = ImageGraphDataset(val_images, val_labels)
# test_dataset  = ImageGraphDataset(test_images, test_labels)
```

ImageGraphDataset: turns a list of (image, label) pairs into a PyG-compatible dataset of graphs.

6.3 Step 7 in Code: GCN Model

A simple 3-layer GCN for graph-level classification, ending with global mean pooling and a linear classifier head.

```
import torch.nn.functional as F
from torch_geometric.nn import GCNConv, global_mean_pool

class GCNClassifier(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, num_classes):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden_channels)
        self.conv2 = GCNConv(hidden_channels, hidden_channels)
        self.conv3 = GCNConv(hidden_channels, hidden_channels)
        self.lin = torch.nn.Linear(hidden_channels, num_classes)

    def forward(self, x, edge_index, batch):
        x = F.relu(self.conv1(x, edge_index))
        x = F.relu(self.conv2(x, edge_index))
        x = F.relu(self.conv3(x, edge_index))
        x = global_mean_pool(x, batch)       # one vector per graph
        x = F.dropout(x, p=0.5, training=self.training)
        return self.lin(x)
```

GCNClassifier: in_channels must equal the number of node features F (5 in the example above: 3 Lab color values + 2 centroid coordinates).

6.4 Steps 8–9 in Code: Training Loop

```
from torch_geometric.loader import DataLoader

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_dataset, batch_size=32)

model = GCNClassifier(in_channels=5, hidden_channels=64,
                      num_classes=NUM_CLASSES).to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=5e-4)
criterion = torch.nn.CrossEntropyLoss()
```

```
def train_one_epoch():
    model.train()
    total_loss = 0
    for batch in train_loader:
        batch = batch.to(device)
        optimizer.zero_grad()
        out = model(batch.x, batch.edge_index, batch.batch)
        loss = criterion(out, batch.y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * batch.num_graphs
    return total_loss / len(train_dataset)

@torch.no_grad()
def evaluate(loader):
    model.eval()
    correct = 0
    for batch in loader:
        batch = batch.to(device)
        out = model(batch.x, batch.edge_index, batch.batch)
        pred = out.argmax(dim=1)
        correct += (pred == batch.y).sum().item()
    return correct / len(loader.dataset)

for epoch in range(1, 51):
    train_loss = train_one_epoch()
    val_acc = evaluate(val_loader)
    print(f'Epoch {epoch:02d} | Train Loss: {train_loss:.4f} | Val Acc: {val_acc:.4f}')
```

Standard PyG training loop. NUM_CLASSES should be set to the number of classes in your dataset.

6.5 Step 11 in Code: Visualizing the Graph

```
import matplotlib.pyplot as plt
from skimage.color import label2rgb

def visualize_graph(image, segments):
    fig, ax = plt.subplots(1, 2, figsize=(10, 5))
    ax[0].imshow(image)
    ax[0].set_title('Original Image')
    ax[0].axis('off')

    ax[1].imshow(label2rgb(segments, image, kind='avg'))
    ax[1].set_title('Superpixel Segmentation (Graph Nodes)')
    ax[1].axis('off')
    plt.tight_layout()
    plt.show()
```

visualize_graph(): shows the original image next to its superpixel segmentation, which corresponds to the graph nodes used by the GNN.

7. Extending the Pipeline to Other Tasks

The graph-construction ideas from Section 5 also apply to other image processing tasks with small modifications, summarized below.

7.1 Semantic Segmentation

- Use the same superpixel graph, but assign one label per superpixel (e.g., the majority ground-truth class among the pixels inside that superpixel), instead of one label per image.
- Train the GNN as a node classifier: remove the global pooling step from the architecture in Section 6.3, and apply CrossEntropyLoss to every node's output.
- To obtain a pixel-level mask, map each predicted node label back onto all pixels belonging to that superpixel.

7.2 Object Detection / Scene Graph Generation

- First run an object detector (e.g., a pretrained Faster R-CNN) to obtain bounding boxes and initial features — each detected object becomes a graph node.
- Create edges between pairs (or all pairs) of detected objects, optionally using their spatial distance or overlap as edge features.
- Use a GNN (GATConv often works well here, since attention can highlight the most relevant object pairs) to refine object class predictions and to classify relationship labels on the edges (e.g., “on”, “holdng”, “next to”).

7.3 Point Clouds / 3D Vision

- Each 3D point becomes a node; edges connect each point to its k nearest neighbors in 3D space.
- Dynamic graph approaches recomputing the k-NN graph at each layer based on the current feature space, as in DGCNN often work well. PyG provides DynamicEdgeConv for this purpose.

8. Common Errors and Debugging Tips

Symptom	Likely Cause / Fix
"RuntimeError: size mismatch" when calling the first GNN layer.	The in_channels of the first GNN layer does not match the number of node features F. Print x.shape and check it equals (N, F).
"Expected all tensors to be on the same device".	Move both the model and each batch to the same device: model.to(device) and batch = batch.to(device).
Pooling / batching errors, or wrong results when using multiple graphs.	Make sure you use torch_geometric.loader.DataLoader (not the plain PyTorch DataLoader) — it correctly batches graphs of different sizes and creates the batch vector needed for pooling.
All predictions collapse to a single class (especially in segmentation).	Usually caused by severe class imbalance. Try a weighted CrossEntropyLoss, oversampling, or a Dice loss.
Out-of-memory errors.	The graph is too large — typically caused by pixel-level graphs. Switch to a superpixel- or patch-based graph (Step 3) to reduce the number of nodes.
Validation accuracy stuck near random guessing.	Check that edges were actually created — print edge_index.shape. An empty or near-empty edge_index means the GNN layers receive almost no neighborhood information.

9. Suggested Datasets and Further Resources

9.1 Documentation and Background Reading

- PyTorch Geometric documentation the official guide for layers, datasets, loaders, and example pipelines.
- scikit-image documentation on the SLIC superpixel algorithm useful for understanding and tuning Step 3.
- NetworkX documentation useful for graph visualization in Step 11.

9.2 Key Papers for Background Reading

- Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks.
- Veličković, P., et al. (2018). Graph Attention Networks.
- Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive Representation Learning on Large Graphs (GraphSAGE).
- Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How Powerful are Graph Neural Networks? (GIN).
- Fey, M., & Lenssen, J. E. (2019). Fast Graph Representation Learning with PyTorch Geometric.
- Achanta, R., et al. (2012). SLIC Superpixels Compared to State-of-the-Art Superpixel Methods.